

Chapter 5

Data Communication Basics

Serial and Parallel Communication

Parallel Communication

In data transmission, **parallel communication** is a method of conveying multiple binary digits (bits) simultaneously. Parallel transmission of data is used for short distance where the speed of information transfer is critical. This form of data communication is found in newer type of computer peripheral equipment with transfer speed of to one million characters per second. The equipment includes printers, disk drives and various other forms of peripheral components.

If the device operates at different speeds, then microprocessor can be used to select a particular speed of operation of the device. The techniques used to transfer data between different speed devices and computer is called synchronizing. There are various ways of synchronization techniques which are involved in parallel data transfer such as simple input and output, simple strobe I/O, single handshaking and double handshaking.

Simple I/O

To get digital data from a simple switch into a microprocessor; switch is connected on input port line from which port can be read. The data is always present and ready so that it can be read at any time. Similarly to output data to a simple display device like LED, the input of LED buffer is connected on an output port pin. And output the logic level required turning on the light. The LED is always there and ready so that data can be sent at any time.

Simple Strobe I/O

In many applications, valid data is present on an external device only at a certain time and must be read in at that time. Here a strobe pulse is supplied to indicate the time at which data is being transmitted. For an example, we can discuss the ASCII encoded keyboard. When a key is pressed, circuitry on keyboard sends out ASCII code for pressed key on eight parallel data lines and then sends out a strobe signal on another line to indicate that valid data is present on eight data lines.

Single Handshaking

Handshaking is the method of synchronizing the actions of slow peripheral devices with that of high speed microprocessor. It can have two transfer schemes.

1. Input Handshake (Peripheral to Microprocessor)

- The peripheral outputs some data and send $\overline{STB}$ signal to microprocessor to tell "Here is the data for you".
- Microprocessor detects asserted $\overline{STB}$ signal, reads the data and sends an acknowledge signal (ACK) to indicate data has been read and peripheral can send next data, "I got that one, send me another".
- Microprocessor sends or receives data when peripheral is ready.

2. Output Handshake (Peripheral from Microprocessor)

Microprocessor outputs data to peripheral and asserts a strobe ($\overline{STB}$) signal. If peripheral is ready it answers back with acknowledgement (ACK) signal to microprocessor.

Double Handshaking

For data transfers where even more coordination is required between the sending system and the receiving system, a double handshake is used. It can have two transfer schemes.

1. Input Handshake (Peripheral to Microprocessor)

- The peripheral asserts its $\overline{STB}$ line low to ask microprocessor "Are you ready?"
- The microprocessor raises its ACK line high to say "I am ready".
- Peripheral then sends data and raises its $\overline{STB}$ line low to say "Here is some valid data for you."
- Microprocessor then reads the data and drops its ACK line to say, "I have the data, thank you, and I await your request to send the next byte of data."

2. Output Handshake(Peripheral from Microprocessor)

Microprocessor sends a strobe ($\overline{STB}$) signal and data and peripheral sends acknowledgement (ACK) signal.

Serial Communication

In data transmission, **serial communication** is the process of sending data one bit at a time, sequentially, over a communication channel or computer bus. Within a microcomputer data is transferred in parallel, because that is the fastest way to do it. For transferring data over long distances, however, parallel data transmission requires too many wires. Therefore, data to be sent long distances is usually converted from parallel form to serial form so that it can be sent on a single wire or pair of wires. Serial data received from a distant source is converted to parallel form so that it can easily be transferred on the microcomputer buses.

• Serial Synchronous Data Transmission

In serial synchronous data transmission, data is transmitted or received based on a clock signal. At a specific rate of data transmission, the transmitting device sends a data bit at each clock pulse. In order to interpret the data correctly, the receiving device must know the start and end of each data unit. The transmitter must know the number of data units to be transferred and the receiver must be synchronized with the data boundaries. Therefore, there must be synchronization between the transmitter and receiver. Usually one or more SYNC characters are used to indicate the start of each synchronous data stream or frame of data. Transmitter sends a large block of data characters one after the other with no time between characters. Transmitting device sends data continuously to the receiving device. If the data is not ready to be transmitted, the line is held in marking condition. To indicate the start of transmission, the transmitter sends out one or more SYNC characters or a unique bit pattern called a flag, depending on the system being used. The receiving device waits for data, when it finds the SYNC characters or the flag then starts interpreting the data which shifts the data following the SYNC characters and converts them to parallel form so they can be read in by a computer.

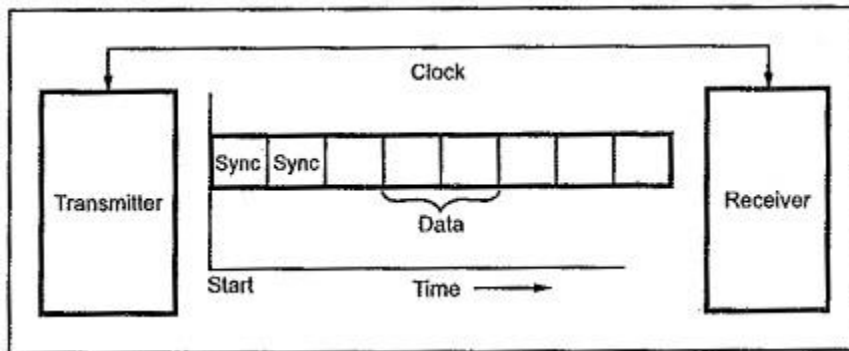


Fig: Synchronous Serial Transmission Format

- Serial Asynchronous Data Transmission

The receiving device does not need to be synchronized with the transmitting device. The transmitting device can send one or more data units when it is ready to send data. Each data unit must be formatted i.e. must contain start and stop bits for indicating beginning and the end of data unit. It also includes one parity bit to identify odd or even parity of data. To send ASCII character, the framing of data should contain:

- 1 start bit: Beginning of data
- 8 bit character: Actual data transferred
- 1 or 2 stop bits: End of data

When no data is being sent, the signal line is in a constant high or marking state. The beginning of the data character is indicated by the line going low for 1 bit time and this bit is called a start bit. The data bits are then sent out on the line one after the other where the least significant bit is sent out first. Parity bit should contain to check for errors in received data. After the data bit and a parity bit, the signal line is returned high for at least 1 bit time to identify the end of the character, this always high bit is referred to as a stop bit. Some older systems use 2 stop bits.

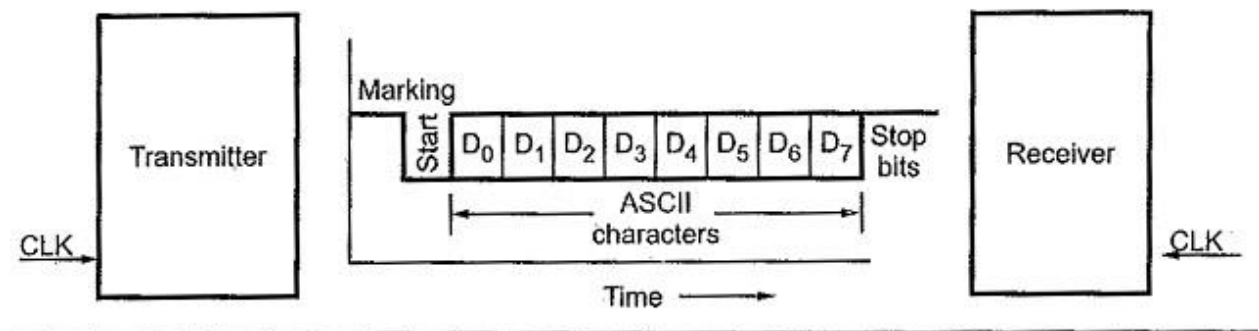


Fig: Serial Asynchronous Data Transmission

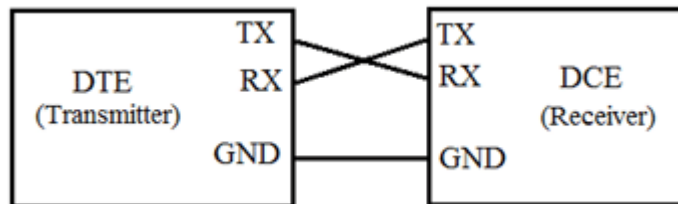
Characteristics of Serial Communication:

- **Baud rate** is used to measure the speed of transmission. It is described as the **number of bits passing in one second**. For example, if the baud rate is 200 then 200 bits per Sec passed. In telephone lines, the baud rates will be **14400, 28800 and 33600**.
- **Stop Bits** are used for a single packet to stop the transmission which is denoted as "T". Some typical values are **1, 1.5 & 2 bits**.
- **Parity Bit** is the simplest form of checking the errors. There are of four kinds, i.e., even odd, marked and spaced. **For example**, If 011 is a number the parity bit=0, i.e., even parity and the parity=1, i.e., odd parity.

RS-232

RS232C "Recommended Standard 232C" is the recent version of Standard 25 pin whereas, RS232D which is of 22 pins. In new PC's male D-type which is of 9 pins.

RS232 is a standard protocol used for serial communication, it is used for connecting computer and its peripheral devices to allow serial data exchange between them. As it obtains the voltage for the path used for the data exchange between the devices. It is used in serial communication up to 50 feet with the rate of 1.492kbps. As EIA defines, the RS232 is used for connecting **Data Transmission Equipment (DTE)** and **Data Communication Equipment (DCE)**.



RS232 protocol

Universal Asynchronous Data Receiver & Transmitter (UART) used in connection with RS232 for transferring data between printer and computer. The microcontrollers are not able to handle such kind of voltage levels, connectors are connected between RS232 signals. These connectors are known as the **DB-9 Connector** as a serial port and they are of two type's **Male connector (DTE) & Female connector (DCE)**.

Electrical Specifications

Let us discuss the electrical specifications of RS232 given below:

- **Voltage Levels:** RS232 also used as ground & 5V level. Binary 0 works with voltages up to +5V to +15Vdc. It is called as 'ON' or spacing (high voltage level) whereas Binary 1 works with voltages up to -5V to -15Vdc. It is called as 'OFF' or marking (low voltage level).
- **Received signal voltage level:** Binary 0 works on the received signal voltages up to +3V to +13 Vdc & Binary 1 works with voltages up to -3V to -13 Vdc.
- **Line Impedances:** The impedance of wires is up to 3 ohms to 7 ohms & the maximum cable length are 15 meters, but new maximum length in terms of capacitance per unit length.

- **Operation Voltage:** The operation voltage will be 250v AC max.
- **Current Rating:** The current rating will be 3 Amps max.
- **Dielectric withstanding voltage:** 1000 VAC min.
- **Slew Rate:** The rate of change of signal levels is termed as Slew Rate. With its slew rate is up to 30 V/microsecond and the maximum bitrate will be 20 kbps.

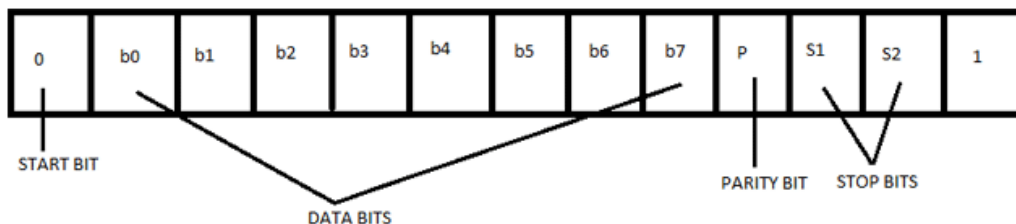
The ratings and specification changes with the change in equipment model.

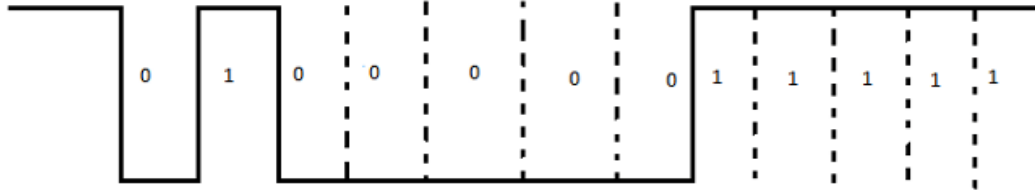
How RS232 Works?

RS232 works on the two-way communication that exchanges data to one another. There are two devices connected to each other, **(DTE) Data Transmission Equipment** & **(DCE) Data Communication Equipment** which has the pins like **TXD, RXD, and RTS & CTS**. Now, from **DTE** source, the **RTS** generates the *request to send* the data. Then from the other side **DCE**, the **CTS**, clears the path for receiving the data. After clearing a path, it will give a signal to **RTS** of the **DTE** source to send the signal. Then the bits are transmitted from **DTE** to **DCE**. Now again from **DCE** source, the request can be generated by **RTS** and **CTS** of **DTE** sources clears the path for receiving the data and gives a signal to send the data. This is the whole process through which data transmission takes place.

TXD	TRANSMITTER
RXD	RECEIVER
RTS	REQUEST TO SEND
CTS	CLEAR TO SEND
GND	GROUND

For example: The signals set to logic 1, i.e., -12V. The data transmission starts from next bit and to inform this, DTE sends start bit to DCE. The start bit is always '0', i.e., +12 V & next 5 to 9 characters is data bits. If we use parity bit, then 8 bits data can be transmitted whereas if parity doesn't use, then 9 bits are being transmitted. The stop bits are sent by the transmitter whose values are 1, 1.5 or 2 bits after the data transmission.



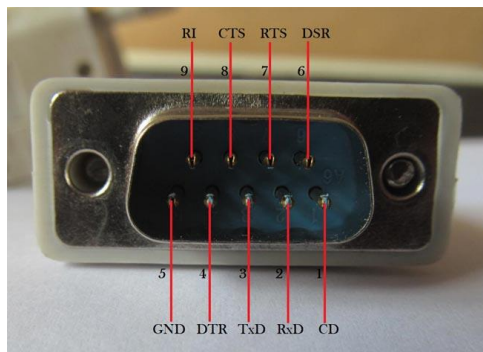


Mechanical Specification

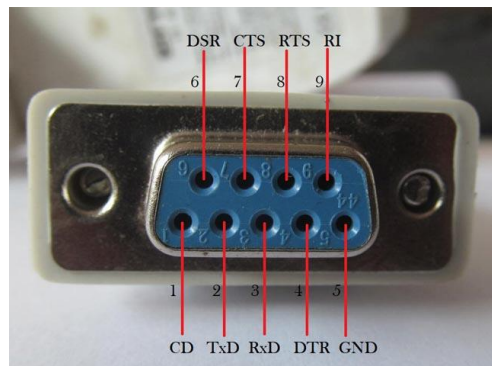
For mechanical specifications, we have to study about two types of connectors that is **DB-25** and **DB-9**. In DB-25, there are 25 pins available which are used for many of the applications, but some of the applications didn't use the whole 25 pins. So, the 9 pin connector is made for the convenience of the devices and equipments.

Now, here we are discussing the **DB-9** pin connector which is used for connection between microcontrollers and connector. These are of two types: **Male Connector (DTE)** & **Female Connector (DCE)**. There are 5 pins on the top row and 4 pins in the bottom row. It is often called **DE-9** or **D-type connector**.

Pin Structure of DB-9 Connector:



RS 232 or DB9 male connector



RS232 or DB9 female connector

Pin Description DB-9 Connector:

PIN No.	Pin Name	Pin Description
1	CD (Carrier Detect)	Incoming signal from DCE

2	RD (Receive Data)	Receives incoming data from DTE
3	TD (Transmit Data)	Send outgoing data to DCE
4	DTR (Data Terminal Ready)	Outgoing handshaking signal
5	GND (Signal ground)	Common reference voltage
6	DSR (Data Set Ready)	Incoming handshaking signal
7	RTS (Request to Send)	Outgoing signal for controlling flow
8	CTS (Clear to Send)	Incoming signal for controlling flow
9	RI (Ring Indicator)	Incoming signal from DCE

Null Modem connection

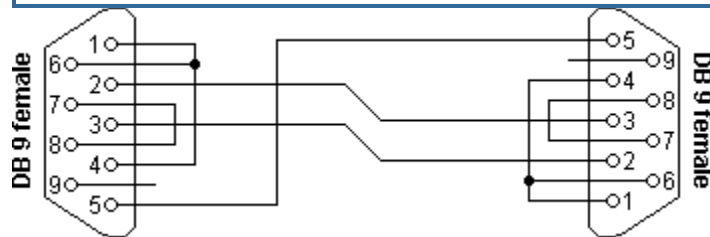
Source: <https://www.lammertbies.nl/comm/info/rs-232-null-modem>

One of the main uses of serial communication today where no modem is involved—a serial null modem configuration with DTE/DTE communication—is not so well defined, especially when it comes to flow control. The terminology null modem for the situation where two computers communicate directly is so often used nowadays, that most people don't realize anymore the origin of the phrase and that a null modem connection is an exception, not the rule.

The simple null modem cable without handshaking shows incompatibilities with common software. The main problem with this cable is that there is a possibility for the software to hang if it checks the modem signal lines in a proper way. I.e. with this null modem cable, good written programs will perform worse than badly written programs.

To overcome this problem and still be able to use a cheap null modem communication cable with only three lines in it, a fake null modem cable layout has been defined. The null modem cable with loop back handshaking resulted from this.

Connector 1	Connector 2	Function
2	3	Rx ◀ Tx
3	2	Tx ▶ Rx
5	5	Signal ground
1 + 4 + 6	–	DTR ▶ CD + DSR
–	1 + 4 + 6	DTR ▶ CD + DSR
7 + 8	–	RTS ▶ CTS
–	7 + 8	RTS ▶ CTS



The main purpose of this null modem cable is to let well defined software think there is handshaking available, with a null modem cable which has no provisions for it.

Compatibility issues

Consider first the DSR signal (pin 6). This input indicates that the other side is ready to start communicating. In the layout, the line is linked back to the DTR output (pin 4). This means, that the software doesn't see the ready signal of the other device, but its own. The same holds for the CD input (pin 1). The assumption is, that if software has been written to check the DSR line to test communication availability, it will probably also set the DTR output to indicate its own state. This is true for at least 99% of all serial communication software. This implies that at least 99% of all serial communication software is capable of faking its own DSR check with this null modem cable.

The same trick is used with the CTS input. In the original use, RTS is set, and then CTS is checked before starting the communication. By setting the RTS output (pin 7) the CTS input on the same connector (pin 8) is receiving clearance immediately. There is no possibility of a software hang-up because of dangling RTS requests.

RS423A

The RS-232 standard defines a bi-directional interface between exactly two communicators, the RS-423 standard defines a uni-directional interface between one transmitter and many receivers. For example, a single computer may need to update a number of terminals that are displaying, for example, customers order information at multiple locations throughout a warehouse. The data direction is always in one direction.

RS-423 allows for distances up to 4000 feet but limits data rates to only 100 kb/sec for a maximum of ten receivers. The voltage levels are +3.6 to +6 volts to represent a binary 0 and -3.6 to -6 volts to represent a binary 1. The voltage levels are defined relative to an earth ground potential assumed to be zero volts. Consequently a difference in ground voltage levels will result in the Common Mode Voltage problem that will confuse the data values.

RS-423 is very similar to RS-422. RS-423 is a serial interface between one DTE and one DCE, unlike RS-422. A DB25 connector is sometimes used in place of the typical DB37, although the DB37 is still used. Like RS-422, all signals use both the A and B lines of a pair, but the B lines in RS-423 are all tied to the Ground (GND). RS-423 is also a single ended signal rather than the balanced signal of RS-422. Most RS-423 signals are synchronous, but asynchronous signals can be found on the MMJ, RJ, DB9, DB15, and 4-wire screw terminals.

RS422A

RS422 was introduced to enable higher data rates to be transferred over serial data lines than was possible with RS232.

RS422 is able to provide data rates of up to 10 Mbps at distances up to 50 feet (15.24 metres). However using reduced data rates, RS422 is able to transmit data over distances of 4000 feet (~1220 metres): the maximum is 100 kbps at this distance.

The key reason why RS422 is able to achieve these improvements results from the use of differential or balanced transmission techniques.

RS422 uses both differential transmitters and receivers which means that it is much more resilient to common mode interference, a key issue with long lines.

Lower voltage line levels are used for RS422: A space is represented by a line voltage level in the band between +2 and +6 volts while a mark is represented by a voltage in the range -2 to -6 volts. The range between +2 and -2 volts provides a good noise margin for the system. Additionally the RS422 standard allows for line impedances down to 50 ohms while supporting the high data rates.

To enable the differential driver to be used, the RS-422 standard uses a four conductor cable. Additionally up to ten receivers can be placed on a single cable, providing a multi-point network or bus.

Although RS422 is significantly different to RS232, it can often be used as a direct interface in many instances.

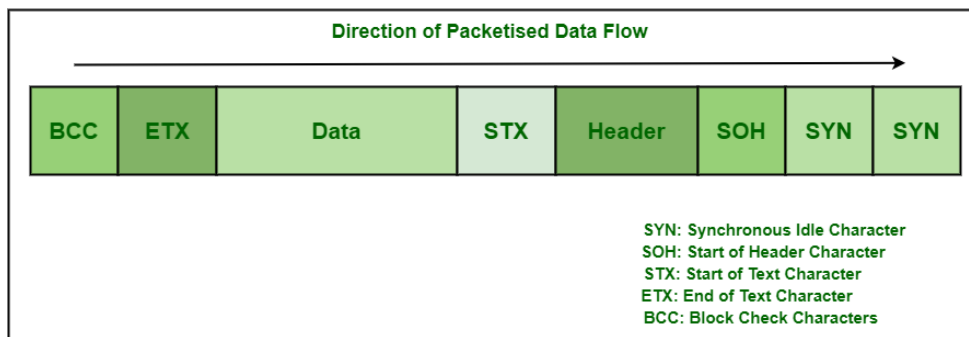
BISYNC Protocol

Bisync is an acronym shortened from "binary synchronous". Bisync is one of the names commonly used when referring to a synchronous communications protocol introduced by IBM back in 1964 with the introduction of a product called the 270X Transmission Control Unit.

Data Frame:

These frames are basically required to carry user data. It is also used to show the direction of data transfer. BISYNC allows for two types of block messages i.e. Single-block and multiple-block message. The only difference in both of these blocks is that data in single-block messages would be terminated with ETB (End of Transmission Block) character rather than an ETX (End of Text) character.

Single Block Data Frame:



Typical Single Block Message